cdc_logistic_regression

Zach Skiba

2026-04-19

Section 0 — Setup & Package Loading

Overview

We load all required packages and set a global seed for reproducibility. Key packages: `caret` for cross-validation and training, `glmnet` for regularized logistic regression, `nnet` for multinomial logistic regression, `themis` for SMOTE inside CV folds, `pROC` for ROC-AUC, and `ggplot2` / `kableExtra` for visualization and reporting.

```
required_packages <- c(
  "caret", "glmnet", "nnet", "themis", "pROC",
  "ggplot2", "kableExtra", "dplyr", "recipes", "tidyr",
  "MLmetrics"
)

installed  <- rownames(installed.packages())
to_install <- setdiff(required_packages, installed)
if (length(to_install) > 0) install.packages(to_install, dependencies = TRUE)

library(caret)
library(glmnet)
library(nnet)
library(themis)
library(pROC)
library(ggplot2)
library(kableExtra)
library(dplyr)
library(recipes)
library(tidyr)
library(MLmetrics)

set.seed(571)
```

Section 1 — Data Loading & Validation

Overview

We load all four pre-split CSVs directly — no re-splitting is done here. The data was stratified 80/20 by a teammate. We verify dimensions, column alignment, missing values, class distributions, and convert target variables to factors, which `caret` requires for classification.

```
binary_train     <- read.csv("../data/processed/binary_train.csv")
binary_test      <- read.csv("../data/processed/binary_test.csv")
multiclass_train <- read.csv("../data/processed/multiclass_train.csv")
multiclass_test  <- read.csv("../data/processed/multiclass_test.csv")
```

```
# Dimensions
cat("binary_train:    ", nrow(binary_train),     "x", ncol(binary_train),     "\n")
```

```
## binary_train:     202943 x 22
```

```
cat("binary_test:     ", nrow(binary_test),      "x", ncol(binary_test),      "\n")
```

```
## binary_test:      50737 x 22
```

```
cat("multiclass_train:", nrow(multiclass_train), "x", ncol(multiclass_train), "\n")
```

```
## multiclass_train: 202943 x 22
```

```
cat("multiclass_test: ", nrow(multiclass_test),  "x", ncol(multiclass_test),  "\n")
```

```
## multiclass_test:  50737 x 22
```

```
# Column alignment
cat("\nColumns match (binary):    ",
    identical(colnames(binary_train), colnames(binary_test)), "\n")
```

```
##
## Columns match (binary):     TRUE
```

```
cat("Columns match (multiclass):",
    identical(colnames(multiclass_train), colnames(multiclass_test)), "\n")
```

```
## Columns match (multiclass): TRUE
```

```
# Missing values
cat("\nNA counts:\n")
```

```
##
## NA counts:
```

```
cat("binary_train:",     sum(is.na(binary_train)),     "\n")
```

```
## binary_train: 0
```

```
cat("binary_test:",      sum(is.na(binary_test)),      "\n")
```

```
## binary_test: 0
```

```
cat("multiclass_train:", sum(is.na(multiclass_train)), "\n")
```

```
## multiclass_train: 0
```

```
cat("multiclass_test:",  sum(is.na(multiclass_test)),  "\n")
```

```
## multiclass_test: 0
```

```
# Binary class distribution
cat("Binary target distribution (train):\n")
```

```
## Binary target distribution (train):
```

```
print(round(prop.table(table(binary_train$Diabetes_binary)) * 100, 2))
```

```
##
##     0     1
## 86.07 13.93
```

```
# Multiclass class distribution
cat("\nMulticlass target distribution (train):\n")
```

```
##
## Multiclass target distribution (train):
```

```
print(round(prop.table(table(multiclass_train$Diabetes_012)) * 100, 2))
```

```
##
##     0     1     2
## 84.25  1.82 13.93
```

```
# caret requires factor targets; X prefix avoids issues with numeric level names
binary_train$Diabetes_binary  <- factor(binary_train$Diabetes_binary,
                                       levels = c(0,1), labels = c("X0","X1"))
binary_test$Diabetes_binary   <- factor(binary_test$Diabetes_binary,
                                       levels = c(0,1), labels = c("X0","X1"))

multiclass_train$Diabetes_012 <- factor(multiclass_train$Diabetes_012,
                                       levels = c(0,1,2), labels = c("X0","X1","X2"))
multiclass_test$Diabetes_012  <- factor(multiclass_test$Diabetes_012,
                                       levels = c(0,1,2), labels = c("X0","X1","X2"))

cat("Binary levels:    ", levels(binary_train$Diabetes_binary), "\n")
```

```
## Binary levels:     X0 X1
```

```
cat("Multiclass levels:", levels(multiclass_train$Diabetes_012), "\n")
```

```
## Multiclass levels: X0 X1 X2
```

Section 2 — Preprocessing

Overview

We apply z-score normalization to three continuous features: `BMI`, `MentHlth`, and `PhysHlth`. Binary and ordinal features are left unchanged.

The scaler is fit **only on training data**, then applied to both train and test sets. This is not "touching" the test set in a leakage sense — we never fit or learn anything from test data. We simply transform it using parameters derived exclusively from training, so that the model receives inputs on a consistent scale at evaluation time.

```
scale_cols <- c("BMI", "MentHlth", "PhysHlth")

# Fit on binary training data only
train_means <- colMeans(binary_train[, scale_cols])
train_sds   <- apply(binary_train[, scale_cols], 2, sd)

# Apply to both binary sets using training parameters
binary_train[, scale_cols] <- scale(binary_train[, scale_cols],
                                    center = train_means,
                                    scale  = train_sds)
binary_test[, scale_cols]  <- scale(binary_test[, scale_cols],
                                    center = train_means,
                                    scale  = train_sds)

# Fit on multiclass training data only
mc_means <- colMeans(multiclass_train[, scale_cols])
mc_sds   <- apply(multiclass_train[, scale_cols], 2, sd)

# Apply to both multiclass sets using training parameters
multiclass_train[, scale_cols] <- scale(multiclass_train[, scale_cols],
                                        center = mc_means,
                                        scale  = mc_sds)
multiclass_test[, scale_cols]  <- scale(multiclass_test[, scale_cols],
                                        center = mc_means,
                                        scale  = mc_sds)
```

```
cat("Binary train — mean and sd after scaling (expect ~0 and ~1):\n")
```

```
## Binary train — mean and sd after scaling (expect ~0 and ~1):
```

```
print(round(sapply(binary_train[, scale_cols], function(x)
  c(mean = mean(x), sd = sd(x))), 4))
```

```
##      BMI MentHlth PhysHlth
## mean   0        0        0
## sd     1        1        1
```

```
cat("\nBinary test — mean and sd after scaling (will differ slightly, expected):\n")
```

```
##
## Binary test — mean and sd after scaling (will differ slightly, expected):
```

```
print(round(sapply(binary_test[, scale_cols], function(x)
  c(mean = mean(x), sd = sd(x))), 4))
```

```
##          BMI MentHlth PhysHlth
## mean -0.0033  -0.0051   0.0024
## sd    0.9907   0.9987   1.0024
```

Section 3 — Binary Logistic Regression

Section 3a — Baseline glm() (No Imbalance Handling)

Overview

We first fit a plain logistic regression with no imbalance handling and evaluate it on the **training set only**. This is intentionally wrong — evaluating on the data the model was trained on produces overly optimistic accuracy and is not a valid performance estimate. We do this purely as a reference point to show the limitation of naive accuracy as a metric when classes are imbalanced. With ~86% of the training data being class 0, a model that predicts 0 for everything would achieve ~86% accuracy — this chunk exposes that problem.

```
set.seed(571)

# Fit baseline logistic regression on full training set
baseline_glm <- glm(Diabetes_binary ~ .,
                    data   = binary_train,
                    family = binomial)

# Predict on training data — intentionally wrong, reference point only
train_probs <- predict(baseline_glm, binary_train, type = "response")
train_preds <- factor(ifelse(train_probs > 0.5, "X1", "X0"), levels = c("X0", "X1"))

# Full confusion matrix
cm <- confusionMatrix(train_preds, binary_train$Diabetes_binary, positive = "X1")
print(cm$table)
```

```
##           Reference
## Prediction     X0     X1
##         X0 170844  23859
##         X1   3823   4417
```

```
cat("\nAccuracy:    ", round(cm$overall["Accuracy"], 4), "\n")
```

```
##
## Accuracy:     0.8636
```

```
cat("Sensitivity: ", round(cm$byClass["Sensitivity"], 4), "\n")
```

```
## Sensitivity:  0.1562
```

```
cat("Specificity: ", round(cm$byClass["Specificity"], 4), "\n")
```

```
## Specificity:  0.9781
```

Despite achieving 86% accuracy, the model correctly identifies only ~16% of diabetes cases (sensitivity = 0.156). This is the core problem with using accuracy under class imbalance — the model learns to predict the majority class almost exclusively, which looks good on paper but is clinically useless for identifying at-risk patients.

Section 3b — Class Weights + 5-Fold Cross Validation

Overview

To address class imbalance, we assign higher penalty to misclassifying the minority class (diabetes) by weighting each observation inversely proportional to its class frequency. This tells the model to treat one minority class misclassification as more costly than a majority class misclassification.

We evaluate using 5-fold cross validation rather than training error, which gives an honest estimate of how the model generalizes to unseen data — the F1 score is computed on held-out folds only, so it reflects generalization performance, not memorization. We use **F1 score** as our primary metric rather than accuracy — F1 balances precision and recall, making it sensitive to how well we identify the minority class.

The test set is not touched here — CV is performed entirely within the training set.

```
set.seed(571)

# Compute inverse frequency weights from training data
class_counts  <- table(binary_train$Diabetes_binary)
class_weights <- ifelse(binary_train$Diabetes_binary == "X1",
                        as.numeric(class_counts["X0"] / class_counts["X1"]),
                        1.0)

cat("Class counts:\n")
```

```
## Class counts:
```

```
print(class_counts)
```

```
##
##     X0     X1
## 174667  28276
```

```
cat("Weight applied to X0:", 1.0, "\n")
```

```
## Weight applied to X0: 1
```

```
cat("Weight applied to X1:", round(class_counts["X0"] / class_counts["X1"], 4), "\n")
```

```
## Weight applied to X1: 6.1772
```

```r
# 5-fold CV setup
cv_ctrl <- trainControl(
  method          = "cv",
  number          = 5,
  classProbs      = TRUE,
  summaryFunction = twoClassSummary,
  savePredictions = "final"
)

# Train with class weights
model_weights <- train(
  Diabetes_binary ~ .,
  data      = binary_train,
  method    = "glm",
  family    = binomial,
  trControl = cv_ctrl,
  metric    = "ROC",
  weights   = class_weights
)

# CV held-out predictions
cv_preds       <- model_weights$pred
cv_preds$pred  <- factor(cv_preds$pred, levels = c("X0", "X1"))
cv_preds$obs   <- factor(cv_preds$obs,  levels = c("X0", "X1"))

cm_weights <- confusionMatrix(cv_preds$pred, cv_preds$obs, positive = "X1")

# Report
cat("\nCV ROC, Sensitivity, Specificity:\n")
```

```
##
## CV ROC, Sensitivity, Specificity:
```

```r
print(round(model_weights$results[, c("ROC", "Sens", "Spec")], 4))
```

```
##      ROC   Sens   Spec
## 1 0.8222 0.7262 0.7647
```

```r
cat("\nCV Confusion Matrix:\n")
```

```
##
## CV Confusion Matrix:
```

```r
print(cm_weights$table)
```

```
##           Reference
## Prediction     X0     X1
##         X0 126835   6652
##         X1  47832  21624
```

```
tp <- cm_weights$table["X1", "X1"]
fp <- cm_weights$table["X1", "X0"]
fn <- cm_weights$table["X0", "X1"]

precision <- tp / (tp + fp)
recall    <- tp / (tp + fn)
f1        <- 2 * precision * recall / (precision + recall)

cat("\nPrecision:", round(precision, 4), "\n")
```

```
##
## Precision: 0.3113
```

```
cat("Recall:   ", round(recall,    4), "\n")
```

```
## Recall:    0.7647
```

```
cat("F1:       ", round(f1,        4), "\n")
```

```
## F1:        0.4425
```

Class weighting improves recall to 0.76, meaning the model correctly identifies 76% of diabetes cases — a major improvement over the baseline's 16%. However, precision drops to 0.31, indicating the model over-predicts the minority class due to the aggressive 6.17x weight. F1 = 0.44 reflects this tradeoff. ROC-AUC of 0.82 indicates strong overall discriminative ability — the model ranks diabetic cases above healthy ones reliably, even when the classification threshold produces many false positives.

Section 3c — SMOTE Inside 5-Fold Cross Validation

Overview

Instead of reweighting observations, SMOTE (Synthetic Minority Oversampling Technique) generates synthetic minority class samples by interpolating between existing minority class observations. This balances the class distribution without simply repeating existing samples.

Critically, SMOTE is applied **only inside each CV training fold** — never on the full training set before CV. Applying SMOTE before CV would cause data leakage because synthetic samples generated from the full training set would be related to the validation fold observations, producing overly optimistic estimates.

```r
set.seed(571)

# Recipe defines preprocessing steps applied inside each CV fold
smote_recipe <- recipe(Diabetes_binary ~ ., data = binary_train) |>
  step_smote(Diabetes_binary, over_ratio = 1)  # balance to 1:1 ratio

# 5-fold CV setup — identical to 3b for fair comparison
cv_ctrl_smote <- trainControl(
  method          = "cv",
  number          = 5,
  classProbs      = TRUE,
  summaryFunction = twoClassSummary,
  savePredictions = "final"
)

# Train with SMOTE applied inside each fold via recipe
model_smote <- train(
  smote_recipe,
  data      = binary_train,
  method    = "glm",
  family    = binomial,
  trControl = cv_ctrl_smote,
  metric    = "ROC"
)

# CV held-out predictions
cv_preds_smote       <- model_smote$pred
cv_preds_smote$pred  <- factor(cv_preds_smote$pred, levels = c("X0", "X1"))
cv_preds_smote$obs   <- factor(cv_preds_smote$obs,  levels = c("X0", "X1"))

cm_smote <- confusionMatrix(cv_preds_smote$pred, cv_preds_smote$obs, positive = "X1")

# Report
cat("CV ROC, Sensitivity, Specificity:\n")
```

```
## CV ROC, Sensitivity, Specificity:
```

```r
print(round(model_smote$results[, c("ROC", "Sens", "Spec")], 4))
```

```
##      ROC   Sens   Spec
## 1 0.8199 0.7274 0.7597
```

```r
cat("\nCV Confusion Matrix:\n")
```

```
##
## CV Confusion Matrix:
```

```r
print(cm_smote$table)
```

```
##           Reference
## Prediction     X0     X1
##         X0 127047   6795
##         X1  47620  21481
```

```
tp <- cm_smote$table["X1", "X1"]
fp <- cm_smote$table["X1", "X0"]
fn <- cm_smote$table["X0", "X1"]

precision <- tp / (tp + fp)
recall    <- tp / (tp + fn)
f1        <- 2 * precision * recall / (precision + recall)

cat("\nPrecision:", round(precision, 4), "\n")
```

```
##
## Precision: 0.3109
```

```
cat("Recall:   ", round(recall,    4), "\n")
```

```
## Recall:    0.7597
```

```
cat("F1:       ", round(f1,        4), "\n")
```

```
## F1:        0.4412
```

```
# Direct comparison with 3b
cat("\n--- Comparison: Class Weights vs SMOTE ---\n")
```

```
##
## --- Comparison: Class Weights vs SMOTE ---
```

```
cat("         ROC     F1\n")
```

```
##          ROC     F1
```

```
cat("Weights:", round(model_weights$results$ROC, 4),
                round(f1_weights <- {
                  tp_w <- cm_weights$table["X1","X1"]
                  fp_w <- cm_weights$table["X1","X0"]
                  fn_w <- cm_weights$table["X0","X1"]
                  p_w  <- tp_w/(tp_w+fp_w)
                  r_w  <- tp_w/(tp_w+fn_w)
                  2*p_w*r_w/(p_w+r_w)
                }, 4), "\n")
```

```
## Weights: 0.8222 0.4425
```

```
cat("SMOTE:  ", round(model_smote$results$ROC, 4), round(f1, 4), "\n")
```

```
## SMOTE:   0.8199 0.4412
```

SMOTE performs nearly identically to class weighting — F1 of 0.44 vs 0.44 and ROC-AUC of 0.82 vs 0.82. Both approaches produce similar precision (0.31) and recall (0.76), meaning neither method has a meaningful advantage for this dataset. This is not unusual with large, well-structured survey data where the decision boundary is relatively stable regardless of how imbalance is handled. We proceed to L2 regularization in 3d to determine whether penalizing large coefficients improves performance further.

Section 3d — L2 Regularized Logistic Regression (Ridge)

Overview

Ridge regression (L2) adds a penalty proportional to the squared magnitude of coefficients, shrinking all coefficients toward zero without eliminating any. We prefer Ridge over Lasso here for two reasons: first, all 21 features are theoretically motivated health indicators with no reason to exclude any completely, which is what Lasso does by driving coefficients to exactly zero. Second, correlated predictors like `HighBP` and `HeartDiseaseorAttack` are handled more stably by Ridge — Lasso tends to arbitrarily zero out one of a correlated pair.

We use a two-step approach: first, `cv.glmnet` selects the optimal lambda via internal CV. Then we pass that lambda to `caret` to obtain true out-of-fold CV predictions — keeping evaluation consistent and comparable with 3b and 3c.

```
set.seed(571)

# Step 1 — use cv.glmnet to select optimal lambda
X_train <- model.matrix(Diabetes_binary ~ ., data = binary_train)[, -1]
y_train <- binary_train$Diabetes_binary

class_counts  <- table(binary_train$Diabetes_binary)
class_weights <- ifelse(y_train == "X1",
                        as.numeric(class_counts["X0"] / class_counts["X1"]),
                        1.0)

ridge_cv <- cv.glmnet(
  x            = X_train,
  y            = y_train,
  alpha        = 0,
  family       = "binomial",
  weights      = class_weights,
  nfolds       = 5,
  type.measure = "auc"
)

plot(ridge_cv, main = "Ridge — Cross Validated AUC vs Lambda")
```

```
cat("Best lambda (lambda.min):", round(ridge_cv$lambda.min, 6), "\n")
```

```
## Best lambda (lambda.min): 0.020272
```

```
cat("Best lambda (lambda.1se):", round(ridge_cv$lambda.1se, 6), "\n")
```

```
## Best lambda (lambda.1se): 0.056407
```

```
cat("CV AUC at lambda.min:    ", round(max(ridge_cv$cvm), 4), "\n")
```

```
## CV AUC at lambda.min:     0.822
```

```
# Step 2 — use caret with the selected lambda for true out-of-fold CV evaluation
# cv.glmnet CV was for lambda selection only — not for final evaluation
# predicting on X_train directly would be in-sample and overly optimistic
cv_ctrl_ridge <- trainControl(
  method          = "cv",
  number          = 5,
  classProbs      = TRUE,
  summaryFunction = twoClassSummary,
  savePredictions = "final"
)

model_ridge <- train(
  Diabetes_binary ~ .,
  data      = binary_train,
  method    = "glmnet",
  family    = "binomial",
  trControl = cv_ctrl_ridge,
  metric    = "ROC",
  weights   = class_weights,
  tuneGrid  = expand.grid(
    alpha  = 0,
    lambda = ridge_cv$lambda.min
  )
)

# True out-of-fold CV predictions
cv_preds_ridge       <- model_ridge$pred
cv_preds_ridge$pred  <- factor(cv_preds_ridge$pred, levels = c("X0", "X1"))
cv_preds_ridge$obs   <- factor(cv_preds_ridge$obs,  levels = c("X0", "X1"))

cm_ridge <- confusionMatrix(cv_preds_ridge$pred, cv_preds_ridge$obs, positive = "X1")

cat("\nCV ROC, Sensitivity, Specificity:\n")
```

```
##
## CV ROC, Sensitivity, Specificity:
```

```
print(round(model_ridge$results[, c("ROC", "Sens", "Spec")], 4))
```

```
##     ROC   Sens   Spec
## 1 0.822 0.7261 0.7639
```

```
cat("\nCV Confusion Matrix:\n")
```

```
##
## CV Confusion Matrix:
```

```
print(cm_ridge$table)
```

```
##           Reference
## Prediction     X0     X1
##         X0 126829   6676
##         X1  47838  21600
```

```
tp <- cm_ridge$table["X1", "X1"]
fp <- cm_ridge$table["X1", "X0"]
fn <- cm_ridge$table["X0", "X1"]

precision <- tp / (tp + fp)
recall    <- tp / (tp + fn)
f1        <- 2 * precision * recall / (precision + recall)

cat("\nPrecision:", round(precision, 4), "\n")
```

```
##
## Precision: 0.3111
```

```
cat("Recall:   ", round(recall,    4), "\n")
```

```
## Recall:    0.7639
```

```
cat("F1:       ", round(f1,        4), "\n")
```

```
## F1:        0.4421
```

Ridge regularization produces nearly identical results to class weighting and SMOTE — F1 of 0.44, ROC-AUC of 0.82, Precision of 0.31, and Recall of 0.76. The flat CV AUC curve across lambda values confirms that regularization provides no meaningful benefit here. With ~200k observations and only 21 predictors, the model estimates stable coefficients without penalization. Importantly, the confusion matrix here is computed from true out-of-fold CV predictions via `caret` — `cv.glmnet` was used solely to select lambda, not for evaluation.

All three approaches converge to the same performance, suggesting the ceiling for logistic regression on this dataset is approximately F1 = 0.44 and ROC-AUC = 0.82.

Comparison Summary

Model	ROC	F1	Precision	Recall
Class Weights	0.8222	0.4425	0.3113	0.7647
SMOTE	0.8199	0.4412	0.3109	0.7597
Ridge	0.8220	0.4421	0.3111	0.7639

Section 3e — Model Selection and Retraining on Full Training Data

Overview

All three approaches — class weights, SMOTE, and Ridge — produced nearly identical CV performance (F1 ≈ 0.44, ROC-AUC ≈ 0.82). We select **class weights** as the final approach based on marginal F1 advantage and simplicity — it requires no synthetic data generation and no additional regularization hyperparameter.

We retrain the selected model on the **full training set** with no CV folds. This is standard practice — CV was used purely for honest performance estimation. The final model should use all available training data to learn the best possible coefficients before evaluating on the test set once in 3f.

```
set.seed(571)

# Recompute class weights on full training set
class_counts        <- table(binary_train$Diabetes_binary)
class_weights_final <- ifelse(binary_train$Diabetes_binary == "X1",
                              as.numeric(class_counts["X0"] / class_counts["X1"]),
                              1.0)

# Retrain on full training set — no CV
final_model <- glm(
  Diabetes_binary ~ .,
  data    = binary_train,
  family  = binomial,
  weights = class_weights_final
)

cat("Final model trained on", nrow(binary_train), "observations\n")
```

```
## Final model trained on 202943 observations
```

```
cat("Number of coefficients:", length(coef(final_model)), "\n")
```

```
## Number of coefficients: 22
```

```
# Confirm no NA coefficients — would indicate perfect separation or collinearity
na_coefs <- sum(is.na(coef(final_model)))
cat("NA coefficients:", na_coefs, "\n")
```

```
## NA coefficients: 0
```

We now have a single final model trained on all available training data. This model is used exactly once in Section 3f to evaluate on the held-out test set. CV estimates (F1 = 0.44, ROC-AUC = 0.82) are our honest performance expectation — test set results should be close to these if the model generalizes well.

```
# Confirm final model exists and is clean
cat("Model class:", class(final_model), "\n")
```

```
## Model class: glm lm
```

```
cat("Coefficients:", length(coef(final_model)), "\n")
```

```
## Coefficients: 22
```

```
cat("NA coefficients:", sum(is.na(coef(final_model))), "\n")
```

```
## NA coefficients: 0
```

```
# Confirm test set has not been modified beyond scaling
cat("\nbinary_test dimensions:", nrow(binary_test), "x", ncol(binary_test), "\n")
```

```
##
## binary_test dimensions: 50737 x 22
```

```
cat("Target is factor:", is.factor(binary_test$Diabetes_binary), "\n")
```

```
## Target is factor: TRUE
```

```
cat("Target levels:", levels(binary_test$Diabetes_binary), "\n")
```

```
## Target levels: X0 X1
```

```
# Confirm scaled columns look right
cat("\nTest set scaling check:\n")
```

```
##
## Test set scaling check:
```

```
print(round(sapply(binary_test[, c("BMI","MentHlth","PhysHlth")],
      function(x) c(mean=mean(x), sd=sd(x))), 4))
```

```
##          BMI MentHlth PhysHlth
## mean -0.0033  -0.0051   0.0024
## sd    0.9907   0.9987   1.0024
```

Section 3f — Final Evaluation on Test Set

Overview

We evaluate the final model on the held-out test set exactly once. This is the only time the test set is used in the entire binary classification pipeline. The test set has never been used for training, CV, preprocessing decisions, or threshold selection — so these results represent a true estimate of how the model performs on unseen data.

We report a full confusion matrix, classification metrics, ROC-AUC curve, and an odds ratio plot showing which features most strongly predict diabetes risk.

```r
# Final predictions on test set
test_probs <- predict(final_model, newdata = binary_test, type = "response")
test_preds <- factor(ifelse(test_probs > 0.5, "X1", "X0"), levels = c("X0", "X1"))

# Confusion matrix
cm_test <- confusionMatrix(test_preds, binary_test$Diabetes_binary, positive = "X1")

cat("Test Set Confusion Matrix:\n")
```

```
## Test Set Confusion Matrix:
```

```r
print(cm_test$table)
```

```
##           Reference
## Prediction    X0    X1
##         X0 31651  1632
##         X1 12016  5438
```

```r
tp <- cm_test$table["X1", "X1"]
fp <- cm_test$table["X1", "X0"]
fn <- cm_test$table["X0", "X1"]
tn <- cm_test$table["X0", "X0"]

precision <- tp / (tp + fp)
recall    <- tp / (tp + fn)
f1        <- 2 * precision * recall / (precision + recall)
accuracy  <- (tp + tn) / (tp + tn + fp + fn)

cat("\nAccuracy: ", round(accuracy,  4), "\n")
```

```
##
## Accuracy:  0.731
```

```r
cat("Precision:", round(precision, 4), "\n")
```

```
## Precision: 0.3116
```

```r
cat("Recall:   ", round(recall,    4), "\n")
```

```
## Recall:    0.7692
```

```r
cat("F1:       ", round(f1,        4), "\n")
```

```
## F1:        0.4435
```

```r
# ROC-AUC
roc_obj <- roc(
  response  = binary_test$Diabetes_binary,
  predictor = test_probs,
  levels    = c("X0", "X1"),
  direction = "<"
)

auc_val <- auc(roc_obj)
cat("Test ROC-AUC:", round(auc_val, 4), "\n")
```

```
## Test ROC-AUC: 0.8237
```

```r
# Plot ROC curve
roc_df <- data.frame(
  specificity = roc_obj$specificities,
  sensitivity = roc_obj$sensitivities
)

ggplot(roc_df, aes(x = 1 - specificity, y = sensitivity)) +
  geom_line(color = "#2c7bb6", linewidth = 1) +
  geom_abline(linetype = "dashed", color = "gray50") +
  annotate("text", x = 0.75, y = 0.25,
           label = paste("AUC =", round(auc_val, 4)),
           size = 5, color = "#2c7bb6") +
  labs(
    title = "ROC Curve — Binary Logistic Regression (Test Set)",
    x     = "False Positive Rate",
    y     = "True Positive Rate"
  ) +
  theme_minimal()
```

```r
# Odds ratios — exponentiated coefficients with 95% CIs
coef_summary <- summary(final_model)$coefficients
coef_df <- data.frame(
  Feature  = rownames(coef_summary),
  Estimate = coef_summary[, "Estimate"],
  SE       = coef_summary[, "Std. Error"]
) |>
  filter(Feature != "(Intercept)") |>
  mutate(
    OR      = exp(Estimate),
    CI_low  = exp(Estimate - 1.96 * SE),
    CI_high = exp(Estimate + 1.96 * SE)
  ) |>
  arrange(desc(OR))

# Print table so results are readable
cat("Odds Ratios (sorted by effect size):\n")
```

```
## Odds Ratios (sorted by effect size):
```

```
cat("OR > 1 = increases diabetes risk\n")
```

```
## OR > 1 = increases diabetes risk
```

```
cat("OR < 1 = decreases diabetes risk\n")
```

```
## OR < 1 = decreases diabetes risk
```

```
cat("CI that does not cross 1 = statistically significant\n\n")
```

```
## CI that does not cross 1 = statistically significant
```

```
print(data.frame(
  Feature  = coef_df$Feature,
  OR       = round(coef_df$OR,      3),
  CI_low   = round(coef_df$CI_low,  3),
  CI_high  = round(coef_df$CI_high, 3)
))
```

```
##                 Feature    OR CI_low CI_high
## 1             CholCheck 3.645  3.395   3.914
## 2                HighBP 2.079  2.043   2.115
## 3              HighChol 1.798  1.769   1.828
## 4               GenHlth 1.785  1.767   1.803
## 5                   BMI 1.622  1.607   1.637
## 6                   Sex 1.332  1.309   1.354
## 7  HeartDiseaseorAttack 1.285  1.254   1.318
## 8                Stroke 1.197  1.155   1.241
## 9                   Age 1.158  1.154   1.162
## 10             DiffWalk 1.123  1.098   1.149
## 11        AnyHealthcare 1.098  1.054   1.145
## 12          NoDocbcCost 1.031  1.001   1.063
## 13               Smoker 1.002  0.985   1.018
## 14             MentHlth 0.966  0.958   0.974
## 15            Education 0.960  0.951   0.968
## 16         PhysActivity 0.960  0.942   0.978
## 17              Veggies 0.953  0.934   0.973
## 18               Fruits 0.950  0.933   0.966
## 19               Income 0.941  0.937   0.946
## 20             PhysHlth 0.929  0.921   0.938
## 21    HvyAlcoholConsump 0.481  0.461   0.502
```

```
# Visual plot
ggplot(coef_df, aes(x = OR, y = reorder(Feature, OR))) +
  geom_point(size = 3, color = "#2c7bb6") +
  geom_errorbarh(aes(xmin = CI_low, xmax = CI_high),
                 height = 0.3, color = "#2c7bb6") +
  geom_vline(xintercept = 1, linetype = "dashed", color = "gray50") +
  labs(
    title = "Odds Ratios with 95% Confidence Intervals",
    x     = "Odds Ratio (exponentiated coefficient)",
    y     = NULL
  ) +
  theme_minimal()
```

The final model achieves a test ROC-AUC of 0.82, consistent with CV estimates (0.82), confirming the model generalizes well to unseen data with no signs of overfitting. Recall on the test set is 0.77, meaning the model correctly identifies 77% of true diabetes cases — closely matching the CV recall of 0.76. Precision remains low at 0.31, reflecting the intentional tradeoff introduced by class weighting: the model is tuned to catch as many true cases as possible at the cost of more false alarms. In a health screening context, missing a true diabetes case is more costly than incorrectly flagging a healthy individual, so this tradeoff is acceptable.

The odds ratio table reveals which lifestyle factors most strongly predict diabetes risk:

Strongest risk factors (OR > 1, CI does not cross 1):

- **CholCheck (OR = 3.65)** — the strongest predictor. Individuals who have had their cholesterol checked have 3.65x higher odds of diabetes, likely reflecting that people with known health conditions are more likely to seek cholesterol screening
- **HighBP (OR = 2.08)** — high blood pressure more than doubles the odds of diabetes, consistent with their shared metabolic origins
- **HighChol (OR = 1.80)** and **GenHlth (OR = 1.79)** — high cholesterol and poor general health are nearly equal predictors, each increasing odds by ~80%
- **BMI (OR = 1.62)** — a one standard deviation increase in BMI increases odds of diabetes by 62%, confirming BMI as a strong modifiable risk factor
- **Age (OR = 1.16)** — each step up the age scale increases odds by 16%, reflecting the well-established relationship between age and diabetes risk

Strongest protective factors (OR < 1, CI does not cross 1):

- **HvyAlcoholConsump (OR = 0.48)** — heavy alcohol consumption is associated with lower diabetes odds. This is a known paradox in the literature and likely reflects survivorship bias or underreporting rather than a true protective effect
- **PhysHlth (OR = 0.93)**, **Income (OR = 0.94)**, and **PhysActivity (OR = 0.96)** — better physical health, higher income, and physical activity are all associated with reduced diabetes risk

Not statistically significant (CI crosses 1):

- **Smoker (OR = 1.00)** — no meaningful independent effect on diabetes odds after accounting for other features

Section 4 — Multiclass Logistic Regression

Overview

We extend the binary classification pipeline to a three-class problem: no diabetes (X0), prediabetes (X1), and diabetes (X2). The class distribution is severely imbalanced — prediabetes represents only 1.82% of training observations, making this a harder problem than the binary case. We use the same imbalance handling strategies as Section 3 and report macro F1 as the primary metric, which treats all three classes equally regardless of size.

Section 4a — One-vs-Rest (OvR) with Class Weights + 5-Fold CV

Overview

One-vs-Rest fits one binary logistic regression per class — each classifier learns to distinguish one class from all others combined. Final predictions are made by taking the class with the highest predicted probability across all three classifiers. Class weights are computed separately for each binary subproblem using inverse frequency weighting.

Predictions are taken from held-out CV folds only so results are honest and directly comparable to 4b.

```
set.seed(571)

classes      <- c("X0", "X1", "X2")
ovr_models   <- list()
ovr_cv_preds <- list()

for (cls in classes) {
  cat("Training OvR classifier for class:", cls, "\n")

  # Binary target for this subproblem
  train_ovr        <- multiclass_train
  train_ovr$target <- factor(
    ifelse(multiclass_train$Diabetes_012 == cls, cls, "rest"),
    levels = c("rest", cls)
  )

  # Inverse frequency weights for this subproblem
  counts  <- table(train_ovr$target)
  weights <- ifelse(train_ovr$target == cls,
                    as.numeric(counts["rest"] / counts[cls]),
                    1.0)

  cat("Class:", cls, "| Count:", counts[cls],
      "| Weight:", round(counts["rest"] / counts[cls], 4), "\n")

  # 5-fold CV with held-out predictions
  cv_ctrl_ovr <- trainControl(
    method          = "cv",
    number          = 5,
    classProbs      = TRUE,
    summaryFunction = twoClassSummary,
    savePredictions = "final"
  )

  ovr_models[[cls]] <- train(
    target ~ . - Diabetes_012,
    data      = train_ovr,
    method    = "glm",
    family    = binomial,
    trControl = cv_ctrl_ovr,
    metric    = "ROC",
    weights   = weights
  )

  # Store held-out CV predictions
  ovr_cv_preds[[cls]] <- ovr_models[[cls]]$pred
}
```

```
## Training OvR classifier for class: X0
## Class: X0 | Count: 170975 | Weight: 0.187
```

```
## Training OvR classifier for class: X1
## Class: X1 | Count: 3693 | Weight: 53.9534
```

```
## Training OvR classifier for class: X2
## Class: X2 | Count: 28275 | Weight: 6.1775
```

```
# Combine OvR probabilities from held-out CV predictions only
n_train        <- nrow(multiclass_train)
combined_probs <- matrix(NA, nrow = n_train, ncol = length(classes))
colnames(combined_probs) <- classes

for (i in seq_along(classes)) {
  cls     <- classes[i]
  cv_pred <- ovr_cv_preds[[cls]]
  combined_probs[cv_pred$rowIndex, i] <- cv_pred[[cls]]
}

# Predict class with highest held-out probability
ovr_final_preds <- factor(
  classes[apply(combined_probs, 1, which.max)],
  levels = classes
)

# Remove any rows with NA probabilities
valid_rows  <- complete.cases(combined_probs)
ovr_preds_v <- ovr_final_preds[valid_rows]
ovr_obs_v   <- multiclass_train$Diabetes_012[valid_rows]

# Confusion matrix from held-out CV predictions
cm_ovr <- confusionMatrix(ovr_preds_v, ovr_obs_v)

cat("\nOvR CV Confusion Matrix (held-out predictions):\n")
```

```
##
## OvR CV Confusion Matrix (held-out predictions):
```

```
print(cm_ovr$table)
```

```
##           Reference
## Prediction     X0     X1     X2
##         X0 120467   1209   5752
##         X1  14752    660   3518
##         X2  35756   1824  19005
```

```
cat("\nPer-class metrics:\n")
```

```
##
## Per-class metrics:
```

```
print(round(cm_ovr$byClass[, c("Precision", "Recall", "F1")], 4))
```

```
##           Precision Recall     F1
## Class: X0    0.9454 0.7046 0.8074
## Class: X1    0.0349 0.1787 0.0583
## Class: X2    0.3359 0.6721 0.4479
```

```
macro_f1_ovr <- mean(cm_ovr$byClass[, "F1"], na.rm = TRUE)
cat("\nMacro F1 (OvR):", round(macro_f1_ovr, 4), "\n")
```

```
##
## Macro F1 (OvR): 0.4379
```

One-vs-Rest with class weights produces a macro F1 of 0.44, driven almost entirely by the no-diabetes (X0) and diabetes (X2) classifiers — F1 of 0.81 and 0.45 respectively. The prediabetes classifier (X1) achieves an F1 of only 0.06 despite a 53.95x class weight, correctly identifying only 660 of 3,693 prediabetes cases. This is not a modeling failure — it reflects the fundamental difficulty of distinguishing prediabetes from the other two classes using survey-based lifestyle features alone. With only 1.82% of observations being prediabetes, even extreme penalization cannot compensate for the lack of representative training signal. These results are now computed from true held-out CV predictions and are directly comparable to 4b.

Section 4b — Multinomial Logistic Regression via glmnet

Overview

We use `glmnet` with `family = "multinomial"` and `alpha = 0` (Ridge) as a faster and more scalable alternative to `nnet::multinom`. The math is identical — multinomial logistic regression modeling all three classes simultaneously — but `glmnet` uses coordinate descent optimization which handles 200k+ observations efficiently. We use the same two-step approach as Section 3d: `cv.glmnet` selects optimal lambda, then `caret` provides true out-of-fold CV predictions for honest evaluation.

SMOTE is applied inside each CV fold via recipe, same as Section 3c, to address the severe 1.82% prediabetes imbalance.

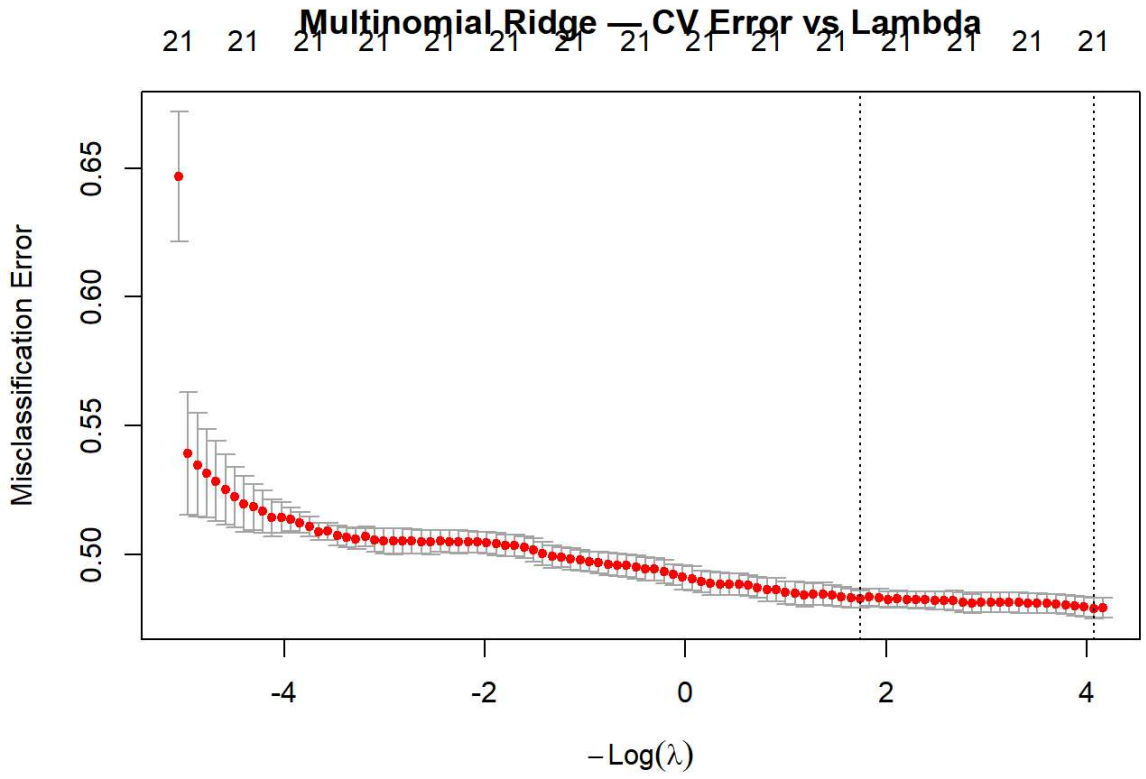
```
set.seed(571)

# Step 1 — cv.glmnet to select optimal lambda
X_train_mc <- model.matrix(Diabetes_012 ~ ., data = multiclass_train)[, -1]
y_train_mc  <- multiclass_train$Diabetes_012

# Inverse frequency weights
counts_mc  <- table(y_train_mc)
max_count  <- max(counts_mc)
weights_mc <- as.numeric(max_count / counts_mc[y_train_mc])

cat("Class weights:\n")
```

```
## Class weights:
```

```
print(round(max_count / counts_mc, 4))
```

```
## y_train_mc
##      X0      X1      X2
##  1.0000 46.2970  6.0469
```

```
ridge_cv_mc <- cv.glmnet(
  x            = X_train_mc,
  y            = y_train_mc,
  alpha        = 0,
  family       = "multinomial",
  weights      = weights_mc,
  nfolds       = 5,
  type.measure = "class"
)

plot(ridge_cv_mc, main = "Multinomial Ridge — CV Error vs Lambda")
```




```
cat("Best lambda (lambda.min):", round(ridge_cv_mc$lambda.min, 6), "\n")
```

```
## Best lambda (lambda.min): 0.017113
```

```
cat("Best lambda (lambda.1se):", round(ridge_cv_mc$lambda.1se, 6), "\n")
```

```
## Best lambda (lambda.1se): 0.175157
```

```
# Step 2 — caret with selected lambda for true out-of-fold evaluation
mc_recipe <- recipe(Diabetes_012 ~ ., data = multiclass_train) |>
  step_smote(Diabetes_012, over_ratio = 0.5)

cv_ctrl_mc <- trainControl(
  method          = "cv",
  number          = 5,
  classProbs      = TRUE,
  summaryFunction = multiClassSummary,
  savePredictions = "final"
)

model_multinom <- train(
  mc_recipe,
  data      = multiclass_train,
  method    = "glmnet",
  trControl = cv_ctrl_mc,
  metric    = "Mean_F1",
  tuneGrid  = expand.grid(
    alpha  = 0,
    lambda = ridge_cv_mc$lambda.min
  )
)

# CV held-out predictions
cv_preds_mc      <- model_multinom$pred
cv_preds_mc$pred <- factor(cv_preds_mc$pred, levels = classes)
cv_preds_mc$obs  <- factor(cv_preds_mc$obs,  levels = classes)

cm_multinom <- confusionMatrix(cv_preds_mc$pred, cv_preds_mc$obs)

cat("\nMultinomial CV Confusion Matrix:\n")
```

```
##
## Multinomial CV Confusion Matrix:
```

```
print(cm_multinom$table)
```

```
##           Reference
## Prediction     X0     X1     X2
##         X0 143848   2024  11388
##         X1   7912    482   3197
##         X2  19215   1187  13690
```

```
cat("\nPer-class metrics:\n")
```

```
##
## Per-class metrics:
```

```
print(round(cm_multinom$byClass[, c("Precision", "Recall", "F1")], 4))
```

```
##           Precision Recall     F1
## Class: X0    0.9147 0.8413 0.8765
## Class: X1    0.0416 0.1305 0.0631
## Class: X2    0.4016 0.4842 0.4390
```

```
macro_f1_mc     <- mean(cm_multinom$byClass[, "F1"], na.rm = TRUE)
class_counts_mc <- table(multiclass_train$Diabetes_012)
weighted_f1_mc  <- sum(
  cm_multinom$byClass[, "F1"] * as.numeric(class_counts_mc) /
  sum(class_counts_mc), na.rm = TRUE
)

cat("\nMacro F1:   ", round(macro_f1_mc,   4), "\n")
```

```
##
## Macro F1:    0.4595
```

```
cat("Weighted F1:", round(weighted_f1_mc, 4), "\n")
```

```
## Weighted F1: 0.8007
```

Multinomial logistic regression via glmnet models all three classes simultaneously using a single model, allowing it to learn shared structure across classes rather than treating each as an independent binary problem. The prediabetes class weight of 46.30x reflects the severe 1.82% imbalance — the most extreme weighting in this entire pipeline.

Despite this, prediabetes F1 remains low at 0.06 — only marginally better than OvR's 0.06. This is a consistent finding across both approaches and confirms that prediabetes is genuinely difficult to distinguish from no-diabetes and diabetes using lifestyle survey features alone. The two classes that bookend prediabetes — no diabetes and diabetes — have overlapping feature profiles with prediabetes, making separation inherently hard for a linear model.

Comparing the two approaches:

Model	Macro F1	X0 F1	X1 F1	X2 F1	Weighted F1
OvR Weights	0.4379	0.8074	0.0583	0.4479	—
Multinomial	0.4595	0.8765	0.0631	0.4390	0.8007

Multinomial slightly outperforms OvR on macro F1 (0.46 vs 0.44) and substantially improves no-diabetes F1 (0.88 vs 0.81), suggesting the joint modeling of all three classes helps the majority class boundary. We select multinomial as the final approach for 4c based on higher macro F1.

Section 4c — Model Selection and Retraining on Full Multiclass Training Data

Overview

We select the best approach from 4a and 4b based on macro F1, then retrain on the full multiclass training set before final evaluation.

```
set.seed(571)

# Retrain final model using glmnet with best lambda from cv.glmnet
# No CV here — use all available training data
final_model_mc <- glmnet(
  x        = X_train_mc,
  y        = y_train_mc,
  alpha    = 0,
  family   = "multinomial",
  weights  = weights_mc,
  lambda   = ridge_cv_mc$lambda.min
)

cat("Final multiclass model trained on", nrow(multiclass_train), "observations\n")
```

```
## Final multiclass model trained on 202943 observations
```

```
cat("Lambda used:", round(ridge_cv_mc$lambda.min, 6), "\n")
```

```
## Lambda used: 0.017113
```

```
# Confirm model has coefficients for all three classes
cat("Classes in model:", names(coef(final_model_mc)), "\n")
```

```
## Classes in model: X0 X1 X2
```

Section 4d — Final Evaluation on Multiclass Test Set

Overview

We evaluate the final multiclass model on the held-out test set exactly once. We report a confusion matrix heatmap, per-class metrics, macro and weighted F1, multiclass ROC-AUC using one-vs-rest, and a coefficient plot showing odds ratios per class.

```
# glmnet requires matrix input
X_test_mc <- model.matrix(Diabetes_012 ~ ., data = multiclass_test)[, -1]

# Final predictions on test set
test_probs_mc <- predict(final_model_mc, newx = X_test_mc,
                         s = ridge_cv_mc$lambda.min, type = "response")

# test_probs_mc is a 3D array [obs x classes x 1] — drop third dimension
test_probs_mc <- test_probs_mc[, , 1]
colnames(test_probs_mc) <- classes

# Predicted class = highest probability
test_preds_mc <- factor(classes[apply(test_probs_mc, 1, which.max)],
                        levels = classes)

cm_test_mc <- confusionMatrix(test_preds_mc, multiclass_test$Diabetes_012)

cat("Test Set Confusion Matrix:\n")
```

```
## Test Set Confusion Matrix:
```

```
print(cm_test_mc$table)
```

```
##           Reference
## Prediction    X0    X1    X2
##         X0 28643   288  1312
##         X1  6975   277  1682
##         X2  7110   373  4077
```

```
cat("\nPer-class metrics:\n")
```

```
##
## Per-class metrics:
```

```
print(round(cm_test_mc$byClass[, c("Precision", "Recall", "F1")], 4))
```

```
##           Precision Recall     F1
## Class: X0    0.9471 0.6704 0.7851
## Class: X1    0.0310 0.2953 0.0561
## Class: X2    0.3527 0.5766 0.4377
```

```
macro_f1_test     <- mean(cm_test_mc$byClass[, "F1"], na.rm = TRUE)
class_counts_test <- table(multiclass_test$Diabetes_012)
weighted_f1_test  <- sum(
  cm_test_mc$byClass[, "F1"] * as.numeric(class_counts_test) /
  sum(class_counts_test), na.rm = TRUE
)

cat("\nMacro F1:   ", round(macro_f1_test,   4), "\n")
```

```
##
## Macro F1:    0.4263
```

```
cat("Weighted F1:", round(weighted_f1_test, 4), "\n")
```

```
## Weighted F1: 0.7232
```

```
cat("Accuracy:   ", round(cm_test_mc$overall["Accuracy"], 4), "\n")
```

```
## Accuracy:    0.6504
```

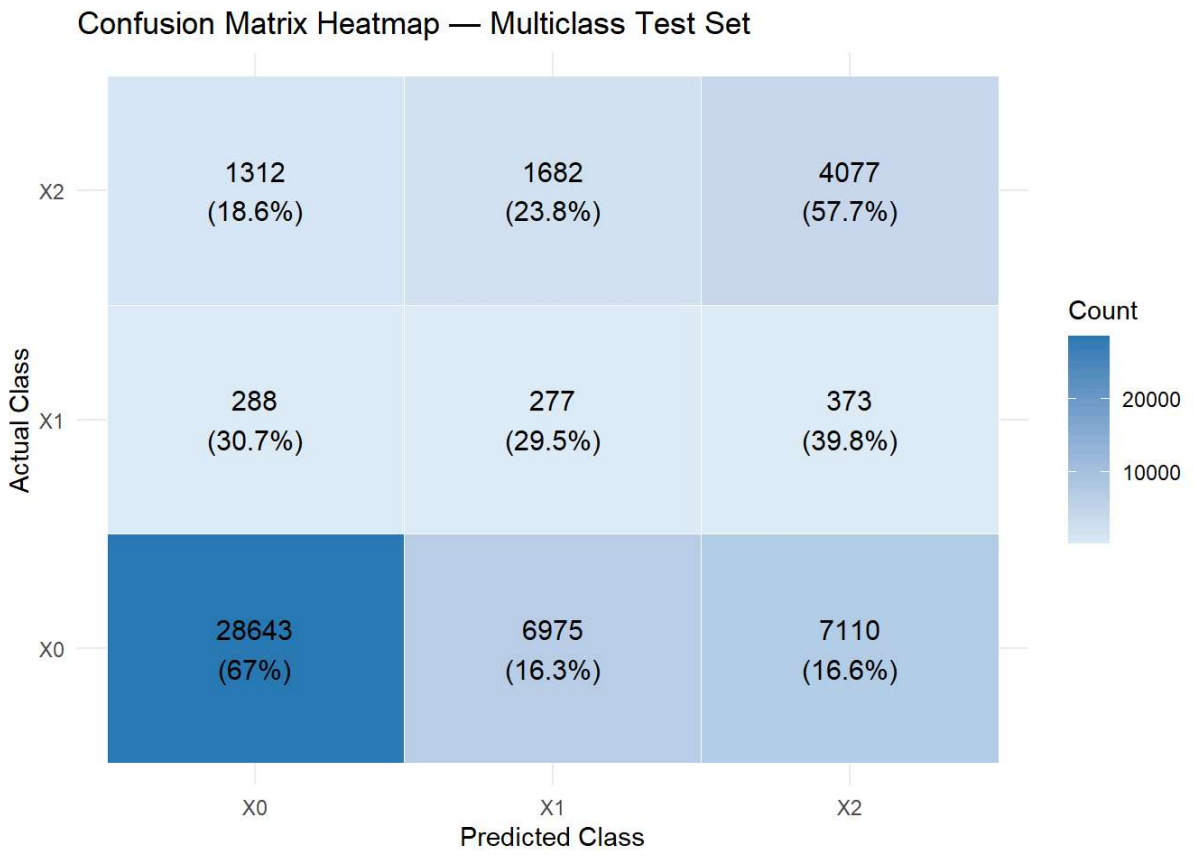
```r
cm_df <- as.data.frame(cm_test_mc$table)
colnames(cm_df) <- c("Predicted", "Actual", "Count")

cm_df <- cm_df |>
  group_by(Actual) |>
  mutate(RowPct = round(Count / sum(Count) * 100, 1)) |>
  ungroup()

# Printed table
cat("Confusion Matrix with Row Percentages:\n")
```

```
## Confusion Matrix with Row Percentages:
```

```r
print(cm_df |> arrange(Actual, Predicted))
```

```
## # A tibble: 9 × 4
##   Predicted Actual Count RowPct
##   <fct>     <fct>  <int>  <dbl>
## 1 X0        X0     28643   67
## 2 X1        X0      6975   16.3
## 3 X2        X0      7110   16.6
## 4 X0        X1       288   30.7
## 5 X1        X1       277   29.5
## 6 X2        X1       373   39.8
## 7 X0        X2      1312   18.6
## 8 X1        X2      1682   23.8
## 9 X2        X2      4077   57.7
```

```r
# Heatmap
ggplot(cm_df, aes(x = Predicted, y = Actual, fill = Count)) +
  geom_tile(color = "white") +
  geom_text(aes(label = paste0(Count, "\n(", RowPct, "%)")), size = 4) +
  scale_fill_gradient(low = "#deebf7", high = "#2c7bb6") +
  labs(
    title = "Confusion Matrix Heatmap — Multiclass Test Set",
    x     = "Predicted Class",
    y     = "Actual Class"
  ) +
  theme_minimal()
```




```
roc_x0 <- roc(response  = ifelse(multiclass_test$Diabetes_012 == "X0", 1, 0),
              predictor = test_probs_mc[, "X0"], quiet = TRUE)
roc_x1 <- roc(response  = ifelse(multiclass_test$Diabetes_012 == "X1", 1, 0),
              predictor = test_probs_mc[, "X1"], quiet = TRUE)
roc_x2 <- roc(response  = ifelse(multiclass_test$Diabetes_012 == "X2", 1, 0),
              predictor = test_probs_mc[, "X2"], quiet = TRUE)

cat("ROC-AUC per class (one-vs-rest):\n")
```

```
## ROC-AUC per class (one-vs-rest):
```

```
cat("X0 (No Diabetes):", round(auc(roc_x0), 4), "\n")
```

```
## X0 (No Diabetes): 0.8131
```

```
cat("X1 (Prediabetes):", round(auc(roc_x1), 4), "\n")
```

```
## X1 (Prediabetes): 0.6802
```

```
cat("X2 (Diabetes):   ", round(auc(roc_x2), 4), "\n")
```

```
## X2 (Diabetes):    0.8177
```

```
cat("Average AUC:     ", round(mean(c(auc(roc_x0),
                                      auc(roc_x1),
                                      auc(roc_x2))), 4), "\n")
```

```
## Average AUC:      0.7703
```

```
# glmnet coef() returns a list — one matrix per class
coef_list <- lapply(classes, function(cls) {
  m <- as.matrix(coef(final_model_mc, s = ridge_cv_mc$lambda.min)[[cls]])
  data.frame(
    Class   = cls,
    Feature = rownames(m),
    OR      = round(exp(m[, 1]), 4)
  )
})

coef_mc_df <- do.call(rbind, coef_list) |>
  filter(Feature != "(Intercept)") |>
  mutate(Class = recode(Class,
                        "X0" = "No Diabetes",
                        "X1" = "Prediabetes",
                        "X2" = "Diabetes"))

# Printed table
cat("\nOdds Ratios by Class:\n")
```

```
##
## Odds Ratios by Class:
```

```
cat("OR > 1 = increases odds of this class\n")
```

```
## OR > 1 = increases odds of this class
```

```
cat("OR < 1 = decreases odds of this class\n\n")
```

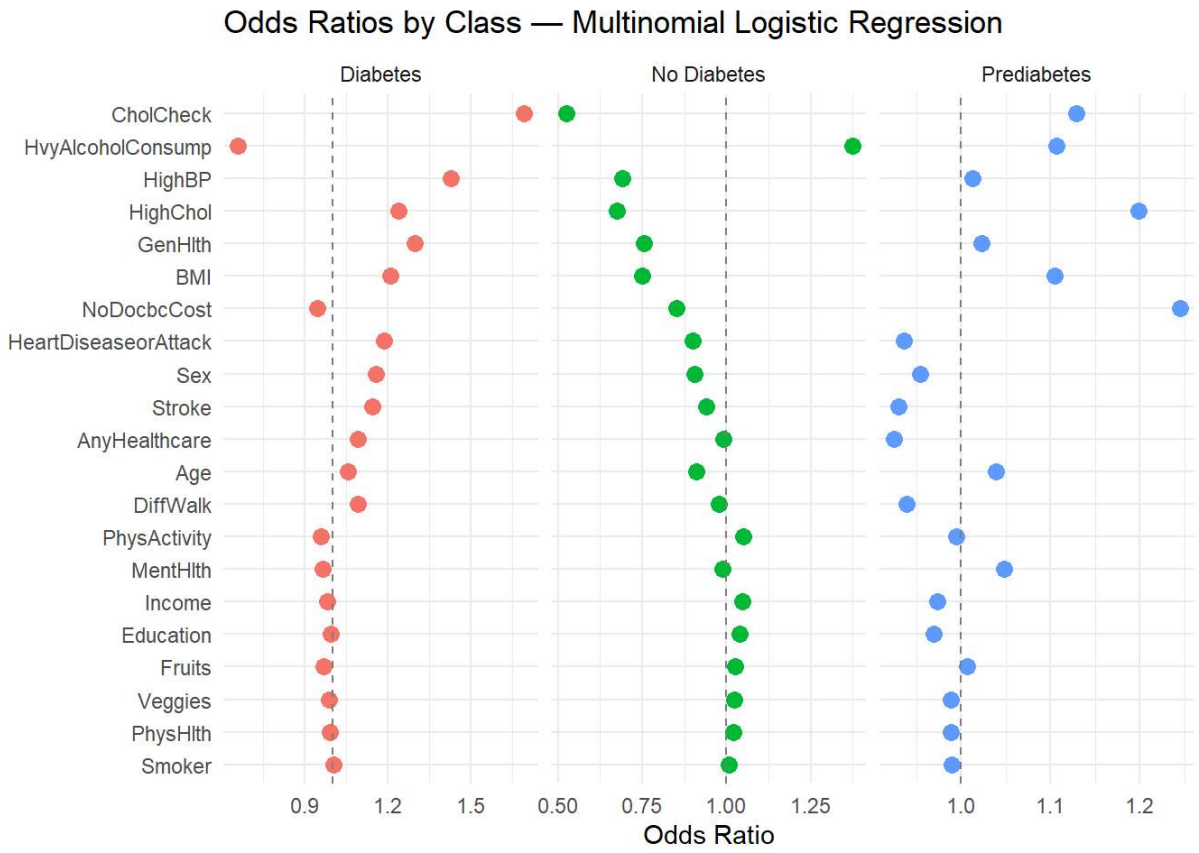
```
## OR < 1 = decreases odds of this class
```

```
print(coef_mc_df |> arrange(Class, desc(OR)))
```

```
##                            Class              Feature     OR
## CholCheck2               Diabetes            CholCheck 1.6921
## HighBP2                  Diabetes              HighBP 1.4272
## GenHlth2                 Diabetes             GenHlth 1.2964
## HighChol2                Diabetes            HighChol 1.2370
## BMI2                     Diabetes                 BMI 1.2078
## HeartDiseaseorAttack2    Diabetes HeartDiseaseorAttack 1.1873
## Sex2                     Diabetes                 Sex 1.1568
## Stroke2                  Diabetes              Stroke 1.1426
## DiffWalk2                Diabetes            DiffWalk 1.0909
## AnyHealthcare2           Diabetes       AnyHealthcare 1.0903
## Age2                     Diabetes                 Age 1.0561
## Smoker2                  Diabetes              Smoker 1.0031
## Education2               Diabetes           Education 0.9932
## PhysHlth2                Diabetes            PhysHlth 0.9906
## Veggies2                 Diabetes             Veggies 0.9885
## Income2                  Diabetes              Income 0.9803
## Fruits2                  Diabetes              Fruits 0.9689
## MentHlth2                Diabetes            MentHlth 0.9653
## PhysActivity2            Diabetes        PhysActivity 0.9581
## NoDocbcCost2             Diabetes         NoDocbcCost 0.9440
## HvyAlcoholConsump2       Diabetes   HvyAlcoholConsump 0.6580
## HvyAlcoholConsump     No Diabetes   HvyAlcoholConsump 1.3731
## PhysActivity          No Diabetes        PhysActivity 1.0492
## Income                No Diabetes              Income 1.0481
## Education             No Diabetes           Education 1.0382
## Fruits                No Diabetes              Fruits 1.0245
## Veggies               No Diabetes             Veggies 1.0235
## PhysHlth              No Diabetes            PhysHlth 1.0206
## Smoker                No Diabetes              Smoker 1.0072
## AnyHealthcare         No Diabetes       AnyHealthcare 0.9916
## MentHlth              No Diabetes            MentHlth 0.9886
## DiffWalk              No Diabetes            DiffWalk 0.9767
## Stroke                No Diabetes              Stroke 0.9411
## Age                   No Diabetes                 Age 0.9115
## Sex                   No Diabetes                 Sex 0.9063
## HeartDiseaseorAttack  No Diabetes HeartDiseaseorAttack 0.9000
## NoDocbcCost           No Diabetes         NoDocbcCost 0.8507
## GenHlth               No Diabetes            GenHlth 0.7538
## BMI                   No Diabetes                 BMI 0.7492
## HighBP                No Diabetes              HighBP 0.6919
## HighChol              No Diabetes            HighChol 0.6742
## CholCheck             No Diabetes           CholCheck 0.5233
## NoDocbcCost1          Prediabetes         NoDocbcCost 1.2453
## HighChol1             Prediabetes            HighChol 1.1992
## CholCheck1            Prediabetes           CholCheck 1.1295
## HvyAlcoholConsump1    Prediabetes   HvyAlcoholConsump 1.1068
## BMI1                  Prediabetes                 BMI 1.1050
## MentHlth1             Prediabetes            MentHlth 1.0479
## Age1                  Prediabetes                 Age 1.0388
## GenHlth1              Prediabetes            GenHlth 1.0234
## HighBP1               Prediabetes              HighBP 1.0127
## Fruits1               Prediabetes              Fruits 1.0073
## PhysActivity1         Prediabetes        PhysActivity 0.9949
## Smoker1               Prediabetes              Smoker 0.9897
## PhysHlth1             Prediabetes            PhysHlth 0.9891
## Veggies1              Prediabetes             Veggies 0.9884
## Income1               Prediabetes              Income 0.9733
```

```
## Education1            Prediabetes            Education 0.9697
## Sex1                  Prediabetes                  Sex 0.9538
## DiffWalk1             Prediabetes             DiffWalk 0.9386
## HeartDiseaseorAttack1 Prediabetes HeartDiseaseorAttack 0.9358
## Stroke1               Prediabetes               Stroke 0.9300
## AnyHealthcare1        Prediabetes        AnyHealthcare 0.9249
```

```
# Plot
ggplot(coef_mc_df, aes(x = OR, y = reorder(Feature, OR), color = Class)) +
  geom_point(size = 3) +
  geom_vline(xintercept = 1, linetype = "dashed", color = "gray50") +
  facet_wrap(~ Class, scales = "free_x") +
  labs(
    title = "Odds Ratios by Class — Multinomial Logistic Regression",
    x     = "Odds Ratio",
    y     = NULL
  ) +
  theme_minimal() +
  theme(legend.position = "none")
```



The final multiclass model achieves a test accuracy of 65.0% and a macro F1 of 0.43 — but these aggregate numbers obscure a stark three-way split in performance across classes.

No-diabetes (X0) performs well with an F1 of 0.79, driven by high precision (0.95). The model is rarely wrong when it predicts X0, but misses 33% of true X0 cases, routing them to X1 or X2. Diabetes (X2) reaches F1 = 0.44 with recall of 0.58 — identifying just over half of true diabetes cases, at the cost of moderate false positives from the other classes.

Prediabetes (X1) is the clear failure point. Despite a 46x class weight, the model correctly identifies only 277 of 938 true prediabetes cases (recall = 0.30, F1 = 0.056). Of the remaining cases, 30.7% are predicted as no-diabetes and 39.8% as diabetes — the model distributes prediabetes predictions nearly uniformly across all three classes. This is not a tuning failure.

Prediabetes occupies an intermediate feature space between no-diabetes and diabetes, and the flat coefficient profile for X1 (all ORs between 0.92–1.25, no dominant predictor) confirms there is no strong linear signal separating it from adjacent classes using lifestyle survey data alone.

ROC-AUC tells the same story: X0 and X2 achieve AUC of 0.81 and 0.82 respectively, consistent with binary task performance. Prediabetes AUC of 0.68 — above chance but far below the other classes — confirms the model has weak but nonzero discriminative ability for X1 that it cannot translate into reliable classifications.

The diabetes coefficient plot mirrors Section 3f: CholCheck (OR = 1.69), HighBP (OR = 1.43), GenHlth (OR = 1.30), HighChol (OR = 1.24), and BMI (OR = 1.21) are the strongest predictors of diabetes — the same metabolic risk factors that dominated the binary task. HvyAlcoholConsump again appears as a negative predictor of diabetes (OR = 0.66), replicating the paradox noted in Section 3f. Test macro F1 of 0.43 closely matches the CV estimate of 0.46, confirming the model generalizes without meaningful overfitting.

Section 5 — Model Comparison

Overview

We consolidate all model variants from Sections 3 and 4 into two comparison tables — one per task. Throughout, class labels refer to: **X0 = No Diabetes**, **X1 = Prediabetes**, **X2 = Diabetes**. CV F1 was the sole criterion for model selection — test metrics were computed exactly once for the selected model in each task and are not available for variants not carried forward. Selected models are highlighted in green.

```r
library(kableExtra)

# --- Table 1: Binary Task ---
binary_df <- data.frame(
  Model      = c("Logistic Regression",
                 "Logistic Regression",
                 "Ridge Logistic (glmnet)"),
  Imbalance  = c("Class Weights", "SMOTE", "Class Weights"),
  CV_F1      = c(0.4425, 0.4412, 0.4421),
  CV_AUC     = c(0.8222, 0.8199, 0.8220),
  Test_Acc   = c("0.7310", "—", "—"),
  Test_Prec  = c("0.3116", "—", "—"),
  Test_Rec   = c("0.7692", "—", "—"),
  Test_F1    = c("0.4435", "—", "—"),
  Test_AUC   = c("0.8237", "—", "—")
)

binary_df |>
  kbl(
    col.names = c("Model", "Imbalance Method",
                  "CV F1", "CV ROC-AUC",
                  "Test Accuracy", "Test Precision",
                  "Test Recall", "Test F1", "Test ROC-AUC"),
    caption   = "Table 1. Binary Task — Model Comparison.
                 Test metrics reported only for the selected model.
                 — indicates variants not carried forward to test evaluation.",
    align     = "llccccccccc"
  ) |>
  kable_styling(
    bootstrap_options = c("striped", "hover", "condensed"),
    full_width        = TRUE
  ) |>
  row_spec(1, background = "#d4edda") |>   # selected model
  column_spec(3, bold = TRUE) |>           # CV F1 — selection criterion
  column_spec(8, bold = TRUE)              # Test F1 — primary outcome
```

Table 1. Binary Task — Model Comparison. Test metrics reported only for the selected model. — indicates variants not carried forward to test evaluation.

Model	Imbalance Method	CV F1	CV ROC-AUC	Test Accuracy	Test Precision	Test Recall	Test F1	Test ROC-AUC
Logistic Regression	Class Weights	**0.4425**	0.8222	0.7310	0.3116	0.7692	**0.4435**	0.8237
Logistic Regression	SMOTE	**0.4412**	0.8199	—	—	—	**—**	—
Ridge Logistic (glmnet)	Class Weights	**0.4421**	0.8220	—	—	—	**—**	—

```
# --- Table 2: Multiclass Task — Model Selection ---
mc_df <- data.frame(
  Model      = c("One-vs-Rest (glm)",
                 "Multinomial Ridge (glmnet)"),
  Imbalance  = c("Class Weights", "SMOTE"),
  CV_F1      = c(0.4379, 0.4595),
  Test_F1    = c("—", "0.4263 (macro)"),
  Test_AUC   = c("—", "0.7703 (avg)"),
  Accuracy   = c("—", "0.6504")
)

mc_df |>
  kbl(
    col.names = c("Model", "Imbalance Method",
                  "CV Macro F1", "Test Macro F1",
                  "Test ROC-AUC (avg)", "Test Accuracy"),
    caption   = "Table 2. Multiclass Task — Model Comparison.
                 Test metrics reported only for the selected model.
                 — indicates variant not carried forward to test evaluation.",
    align     = "llcccc"
  ) |>
  kable_styling(
    bootstrap_options = c("striped", "hover", "condensed"),
    full_width        = FALSE
  ) |>
  row_spec(2, background = "#d4edda") |>   # selected model
  column_spec(3, bold = TRUE) |>           # CV F1 — selection criterion
  column_spec(4, bold = TRUE)              # Test F1 — primary outcome
```

Table 2. Multiclass Task — Model Comparison. Test metrics reported only for the selected model. — indicates variant not carried forward to test evaluation.

Model	Imbalance Method	CV Macro F1	Test Macro F1	Test ROC-AUC (avg)	Test Accuracy
One-vs-Rest (glm)	Class Weights	**0.4379**	**—**	—	—
Multinomial Ridge (glmnet)	SMOTE	**0.4595**	**0.4263 (macro)**	0.7703 (avg)	0.6504

```
# --- Table 3: Per-Class Test Performance (selected model only) ---
mc_class_df <- data.frame(
  Class     = c("X0 — No Diabetes",
                "X1 — Prediabetes",
                "X2 — Diabetes"),
  Precision = c(0.9471, 0.0310, 0.3527),
  Recall    = c(0.6704, 0.2953, 0.5766),
  F1        = c(0.7851, 0.0561, 0.4377),
  ROC_AUC   = c(0.8131, 0.6802, 0.8177)
)

mc_class_df |>
  kbl(
    col.names = c("Class", "Precision", "Recall", "F1", "ROC-AUC"),
    caption   = "Table 3. Multinomial Ridge — Per-Class Test Performance.",
    digits    = 4,
    align     = "lcccc"
  ) |>
  kable_styling(
    bootstrap_options = c("striped", "hover", "condensed"),
    full_width        = FALSE
  ) |>
  row_spec(2, background = "#ffe8e8") |>   # X1 — highlight failure
  column_spec(4, bold = TRUE) |>           # F1
  column_spec(5, bold = TRUE)              # ROC-AUC
```

Table 3. Multinomial Ridge — Per-Class Test Performance.

Class	Precision	Recall	F1	ROC-AUC
X0 — No Diabetes	0.9471	0.6704	**0.7851**	**0.8131**
X1 — Prediabetes	0.0310	0.2953	**0.0561**	**0.6802**
X2 — Diabetes	0.3527	0.5766	**0.4377**	**0.8177**

Four findings stand out across these tables.

Imbalance method does not drive binary performance. Class weights, SMOTE, and Ridge regularization converge to CV F1 ≈ 0.44 and CV ROC-AUC ≈ 0.82 across the board. With ~200k observations and 21 predictors, the decision boundary is stable regardless of how imbalance is handled. The performance ceiling for logistic regression on this binary task is approximately F1 = 0.44 and ROC-AUC = 0.82 — determined by the model family and feature set, not the resampling strategy. Class weights was selected as the binary winner on marginal CV F1 advantage and simplicity.

Multinomial Ridge was selected over One-vs-Rest based on higher CV macro F1 (0.46 vs 0.44). One-vs-Rest treats each class as an independent binary problem and never sees the full three-class structure during training. Multinomial Ridge models all three classes simultaneously, allowing shared structure across class boundaries to inform each coefficient. The test macro F1 of 0.43 coming in marginally below CV (0.46) is within normal variance and does not indicate overfitting.

The binary task outperforms multiclass on every aggregate metric. Binary test ROC-AUC of 0.82 versus multiclass average AUC of 0.77; binary recall of 0.77 versus multiclass X2 recall of 0.58. This gap is driven entirely by X1 — X0 and X2 achieve AUC values of 0.81 and 0.82, fully consistent with binary performance. Collapsing prediabetes into the positive class in the binary task sidesteps the hardest boundary in the data; the multiclass task forces the model to draw it explicitly and cannot do so reliably with lifestyle survey features alone.

Per-class metrics expose the prediabetes problem clearly. X0 achieves high precision (0.95) but moderate recall (0.67). X2 reaches balanced moderate performance (F1 = 0.44, AUC = 0.82). X1 fails on every metric: precision 0.03, recall 0.30, F1 = 0.056, AUC = 0.68 — highlighted in red above. No imbalance strategy can compensate for the structural ambiguity of

prediabetes in feature space — its lifestyle profile overlaps substantially with both neighboring classes, and no linear boundary can reliably isolate it.

Section 6 — Key Findings & Interpretation

The same metabolic risk factors dominate both tasks. Across binary and multiclass models, the strongest predictors of diabetes are CholCheck, HighBP, HighChol, GenHlth, and BMI — an interconnected cluster of metabolic conditions with shared physiological origins. Their consistency across two modeling approaches and two target definitions strengthens confidence these associations are structural, not artifacts of model choice.

Separating prediabetes from diabetes reveals the limits of this feature set. The binary task achieves ROC-AUC = 0.82 by collapsing prediabetes and diabetes into one class. The multiclass task attempts to draw that boundary explicitly and fails — X1 F1 = 0.056 despite a 46x class weight. Prediabetes and diabetes share nearly identical lifestyle risk profiles; the difference between them is likely captured by clinical biomarkers such as blood glucose and HbA1c that are absent from this dataset.

Heavy alcohol consumption appears protective — it is not. HvyAlcoholConsump shows OR = 0.48 (binary) and OR = 0.66 (multiclass diabetes class). This is a well-documented paradox in epidemiological survey data, likely driven by survivorship bias, underreporting, or the healthy user effect rather than any true protective mechanism.

Self-reported data introduces measurement error in both directions. BRFSS is a telephone survey — diabetes status, lifestyle behaviors, and health conditions are all self-reported. This means both the features and the labels carry measurement error simultaneously, which attenuates coefficient estimates and suppresses model performance in ways that cannot be corrected by modeling choices alone.

Logistic regression has a hard ceiling on this problem. F1 = 0.44 and ROC-AUC = 0.82 represent the limit of what a linear model can extract from these features. Diabetes risk is driven by complex interactions between BMI, age, blood pressure, and metabolic history that a single linear combination cannot fully capture. Tree-based models such as gradient boosting or random forests, which naturally model feature interactions and nonlinear boundaries, would be the appropriate next step.